

机场航班运行仿真系统

概要设计说明书

Airport Flight Simulation System

项目	内容
项目名称	机场航班运行仿真系统
技术架构	Spring Boot 3.3.5 + Spring MVC + Spring Data JPA + Thymeleaf
数据存储	MySQL 8.x; 本地便携与测试环境使用 H2
文档版本	V1.0
编写日期	2026-06-12

修订记录

版本	日期	说明	编写人
V1.0	2026-06-12	依据需求分析文档和现有项目代码形成概要设计说明书	项目组

1 引言

1.1 编写目的

本文档用于说明“机场航班运行仿真系统”的总体设计方案，明确系统整体架构、模块划分、模块间接口、核心数据结构、数据库设计以及关键业务逻辑。本文档在需求分析说明书的基础上形成代码实现蓝图，为后续详细设计、编码、测试、部署和课程答辩提供依据。

1.2 项目背景

机场航班运行具有资源受限和时序推进特征。航班在计划起飞或降落时间附近需要占用跑道，降落航班还需要匹配停机位；当资源不足时，航班进入等待并累计延误。本系统以课程实验为目标，抽象真实机场运行中的核心资源竞争过程，实现基础数据维护、仿真任务控制、离散时间步调度、事件日志记录和统计展示。

1.3 设计范围

- 航班、跑道、停机位等基础数据的持久化管理。
- 仿真任务创建、单步推进、自动运行和仿真重置。
- 基于计划时间、航班类型和优先级的资源调度逻辑。
- 仿真事件日志、仪表盘、统计结果、运行洞察和数据导入导出页面。
- MySQL 正式运行模式、H2 本地便携模式和 Docker 部署模式。

2 总体设计

2.1 设计目标

- 业务闭环完整：覆盖基础数据录入、仿真启动、时间推进、资源分配、状态变化、日志记录和统计展示。
- 架构清晰：采用 Controller、Service、Repository、Entity 分层，页面交互与业务规则解耦。

- 规则可解释：调度算法以先到先服务为基础，进港优先并结合业务优先级，便于演示和测试。
- 数据可追溯：所有关键调度事件写入 `simulation_record`，支持按任务、航班和事件类型查询。
- 环境可迁移：支持 MySQL 生产配置、H2 本地模式和 Docker Compose 部署。

2.2 系统架构

系统采用典型三层 Web 应用架构，页面请求经 Spring MVC Controller 进入 Service 业务层，再通过 Spring Data JPA 访问数据库。

浏览器 / Thymeleaf 页面



Controller 层：页面路由、表单接收、模型组装、重定向



Service 层：参数校验、仿真调度、统计计算、导入导出、事务控制



Repository 层：JPA 查询、筛选、排序、持久化



MySQL / H2: `flight`、`runway`、`gate`、`simulation_task`、`simulation_record`

2.3 技术架构

层次	技术/组件	设计说明
表示层	Thymeleaf、HTML、CSS、JavaScript	提供登录页、仪表盘、业务管理、仿真运行、日志和统计页面。
控制层	Spring MVC Controller	接收页面请求，调用业务服务，向模板传递 Model 数据。
业务层	Spring Service、事务管理	封装 CRUD 校验、仿真推进、调度算法、统计聚合和运行洞察。
数据访问层	Spring Data JPA	通过 Repository 完成实体持久化和条件查询。

数据层	MySQL 8.x、H2	MySQL 用于正式运行；H2 用于本地便携启动和自动化测试。
构建与部署	Maven、Docker、Docker Compose	支持本地打包运行和服务容器化部署。

3 模块设计

3.1 功能模块划分

模块	主要页面/入口	职责
登录与首页仪表盘模块	/、/login、 /dashboard	提供系统入口、核心统计、资源状态和近期事件概览。
航班管理模块	/flights	维护航班计划，支持新增、编辑、删除保护、组合筛选和状态展示。
跑道管理模块	/runways	维护跑道编号、状态、支持类型和当前占用信息。
停机位管理模块	/gates	维护停机位编号、适配机型、状态和占用时间。
仿真运行模块	/simulation	创建任务、单步推进、自动运行、重置仿真，展示队列和资源状态。
仿真日志模块	/records	按任务、航班、事件类型筛选仿真事件。
统计结果模块	/statistics	聚合航班完成率、延误率、平均延误和资源占用率。
运行洞察模块	/operations/*	提供冲突查看、事件看板、运行计划、CSV 导入导出、场景对比和角色工作台。

3.2 代码包结构

包名	内容	说明
controller	DashboardController、 FlightController 等	页面控制器，负责路由映射和视图模型组装。
service	SimulationService、	核心业务逻辑、事务边界和统计

	StatisticsService 等	计算。
repository	FlightRepository、 RunwayRepository 等	JPA 数据访问接口和条件查询。
entity	Flight、Runway、Gate、 SimulationTask、 SimulationRecord	数据库表对应的领域实体。
enums	FlightStatus、ResourceStatus 等	系统状态、类型和事件枚举。
templates/static	Thymeleaf 页面和静态资源	前端页面、样式、脚本和图片资源。

4 模块间接口设计

4.1 控制层接口

接口	方法	控制器	功能说明
/dashboard	GET	DashboardController	展示统计摘要、航班、跑道、停机位、近期日志和最新任务。
/flights	GET/POST	FlightController	航班列表筛选、新增和保存。
/flights/{id}	GET/POST	FlightController	航班编辑、更新和删除。
/runways	GET/POST	RunwayController	跑道列表、新增和保存。
/gates	GET/POST	GateController	停机位列表、新增和保存。
/simulation/tasks	POST	SimulationController	创建仿真任务。
/simulation/tasks/{id}/step	POST	SimulationController	推进一个时间步。
/simulation/tasks/{id}/run	POST	SimulationController	自动运行到仿真结束。
/records	GET	RecordController	按条件查看仿真日志。
/operations/export/*.csv	GET	OperationsController	导出仿真日志和统计摘

			要 CSV。
--	--	--	--------

4.2 业务层接口

服务类	核心方法	输入/输出	职责
SimulationService	createTask、step、runAll、reset	SimulationTask、taskId	仿真任务生命周期与调度主流程。
FlightService	findAll、save、deleteById	筛选条件、Flight	航班查询、校验、保存和删除保护。
RunwayService	findAll、save、deleteById	Runway	跑道基础数据维护与占用状态保护。
GateService	findAll、save、deleteById	Gate	停机位基础数据维护与占用状态保护。
StatisticsService	getSummary	无输入，返回StatisticsSummary	聚合航班和资源统计指标。
OperationsInsightService	findConflicts、importFlights、exportRecordsCsv	CSV 文件、查询结果	运行洞察、计划冲突、导入导出与场景数据。

5 数据结构设计

5.1 核心实体

实体	关键字段	设计说明
Flight	flightNo、flightType、aircraftType、planned/actual time、status、delayMinutes、priority	描述航班计划、运行结果、状态和调度优先级。
Runway	runwayNo、status、supportType、currentFlightId、occupiedStart/EndTime	描述跑道可用性、起降支持类型和占用窗口。
Gate	gateNo、status、aircraftType、	描述停机位适配机型、状态和

	currentFlightId、 occupiedStart/EndTime	占用窗口。
SimulationTask	taskName、startTime、endTime、 timeStepMinutes、currentTime、 status	描述一次仿真的时间范围、步 长和当前进度。
SimulationRecord	taskId、flightId、eventTime、 eventType、description	记录每次入队、分配、延误、 释放和完成事件。

5.2 枚举设计

枚举	取值	用途
FlightType	DEPARTURE、ARRIVAL	区分起飞航班和降落航班。
AircraftType	SMALL、MEDIUM、LARGE	匹配停机位适配机型。
FlightStatus	SCHEDULED、WAITING、 DELAYED、COMPLETED、 CANCELLED	表示航班调度状态。
ResourceStatus	IDLE、OCCUPIED、 MAINTENANCE	表示跑道和停机位资源状态。
RunwaySupportType	TAKEOFF_ONLY、 LANDING_ONLY、BOTH	表示跑道支持的操作类型。
SimulationTaskStatus	CREATED、RUNNING、 FINISHED	表示仿真任务生命周期。
SimulationEventType	ENTER_QUEUE、 ASSIGN_RUNWAY、 ASSIGN_GATE、 TAKEOFF、LANDING、 DELAY、 RELEASE_RESOURCE、 COMPLETE	表示仿真日志事件类型。

6 数据库设计

6.1 表结构概要

表名	主键	唯一/索引	说明
flight	id	flight_no 唯一	航班计划与运行结果表。
runway	id	runway_no 唯一	跑道资源表。
gate	id	gate_no 唯一	停机位资源表。
simulation_task	id	按 created_time、id 获取最新任务	仿真任务表。
simulation_record	id	idx_simulation_record_task_time、 idx_simulation_record_flight	仿真事件日志表。

6.2 数据关系

simulation_record.task_id 逻辑关联 simulation_task.id, simulation_record.flight_id 逻辑关联 flight.id; runway.current_flight_id 和 gate.current_flight_id 记录当前占用资源的航班。当前 SQL 以逻辑关联和索引为主, 便于课程演示时重置数据; 后续正式化版本可补充外键约束和级联策略。

6.3 数据约束

- flight.flight_no、runway.runway_no、gate.gate_no 必须唯一。
- simulation_task.end_time 不得早于 start_time, time_step_minutes 必须为正整数。
- 起飞航班必须具备计划起飞时间, 降落航班必须具备计划降落时间。
- 资源释放后必须清空 current_flight_id、occupied_start_time 和 occupied_end_time。
- 仿真记录必须包含 task_id、event_time、event_type 和 description, 保证事件可追踪。

7 核心业务逻辑设计

7.1 仿真任务流程

1. 用户在仿真页面创建任务, 输入任务名称、开始时间、结束时间和时间步长。
2. 系统校验任务参数并初始化 currentTime 为 startTime, 任务状态为 CREATED。
3. 单步推进时, 系统在当前仿真时间释放已到期占用资源。
4. 系统筛选计划时间小于等于 currentTime 且状态为 SCHEDULED、WAITING 或 DELAYED 的航班。

5. 待处理航班按计划时间、进港优先、优先级从高到低排序。
6. 系统尝试为每个航班分配资源；成功则写入分配和完成事件，失败则进入或保持延误状态并累计延误分钟数。
7. `currentTime` 增加 `timeStepMinutes`；若超过 `endTime`，则任务状态置为 `FINISHED`，否则置为 `RUNNING`。

7.2 起飞调度逻辑

起飞航班只需要可用跑道。系统查找状态为 `IDLE` 且 `supportType` 为 `TAKEOFF_ONLY` 或 `BOTH` 的跑道。

if idle runway exists:

 occupy runway for [`currentTime`, `currentTime` + `stepMinutes`]

 set `actualDepartureTime` = `currentTime`

 set `flight.status` = `COMPLETED`

 record `ASSIGN_RUNWAY`, `TAKEOFF`, `COMPLETE`

else:

 set `flight.status` = `DELAYED`

`delayMinutes` += `stepMinutes`

 record `DELAY`

7.3 降落调度逻辑

降落航班同时需要可用跑道和匹配机型的可用停机位；任一资源不足即延误。

if idle landing runway exists and idle gate for `aircraftType` exists:

 occupy runway and gate for [`currentTime`, `currentTime` + `stepMinutes`]

 set `actualArrivalTime` = `currentTime`

 set `flight.status` = `COMPLETED`

 record `ASSIGN_RUNWAY`, `ASSIGN_GATE`, `LANDING`, `COMPLETE`

else:

 set `flight.status` = `DELAYED`

`delayMinutes` += `stepMinutes`

record DELAY

7.4 资源释放逻辑

每次推进前，系统扫描状态为 OCCUPIED 的跑道和停机位。若 occupiedEndTime 不晚于当前仿真时间，则将资源状态恢复为 IDLE，清空占用航班和占用时间，并记录 RELEASE_RESOURCE 事件。

7.5 统计计算逻辑

指标	计算方式	展示用途
总航班数	flight 表记录总数	仪表盘和统计页基础指标。
起飞/降落航班数	按 flightType 计数	展示运行结构。
已完成航班数	status = COMPLETED	计算完成率。
延误航班数	status = DELAYED	计算延误率。
平均延误	delayMinutes 平均值，保留两位小数	衡量整体延误水平。
最大延误	delayMinutes 最大值	定位最严重等待情况。
跑道/停机位占用率	OCCUPIED 数 / 资源总数	展示资源实时占用情况。

8 页面与交互设计

页面	主要内容	关键交互
登录页	系统入口和演示登录表单	提交后进入仪表盘。
仪表盘	统计卡片、航班列表、资源状态、近期日志、最新任务	查看全局运行态势。
航班管理	航班列表、筛选、表单	新增、编辑、删除、按条件筛选。
跑道管理	跑道列表和表单	维护状态、支持类型和占用情况。
停机位管理	停机位列表和表单	维护适配机型、状态和占用情况。
仿真运行	任务创建、队列、资源状态、近期事件	单步推进、自动运行、重置、加载测试场景。

仿真日志	事件记录列表	按任务、航班、事件类型筛选。
统计结果	航班统计、延误统计、资源占用统计	查看仿真输出指标。
运行洞察	冲突、事件、计划、数据、场景、角色页面	导入航班 CSV、导出统计/日志、查看冲突建议。

9 非功能设计

9.1 性能与可用性

- 课程规模下以单机部署为主，数据量较小，JPA 查询和内存排序可满足演示需求。
- 仿真自动运行设置 guard 上限，避免异常任务导致无限循环。
- 关键页面以服务端渲染为主，减少前端复杂度并提升部署可靠性。

9.2 可靠性与一致性

- SimulationService 使用事务边界，保证一次时间步内任务、航班、资源和日志更新的一致性。
- 删除航班、跑道、停机位时由 Service 层执行占用状态和业务约束检查。
- 仿真重置会清空任务和日志，并恢复航班及资源初始状态，便于重复演示。

9.3 可维护性

- Controller 只负责页面交互，调度规则集中在 SimulationService，便于单元测试和规则迭代。
- 枚举集中定义状态和类型，减少魔法字符串。
- Repository 层封装筛选查询，保持业务层代码清晰。

9.4 安全性

当前登录为课程演示入口，未接入 Spring Security。正式环境应补充账号认证、密码加密、权限控制、CSRF 防护、导入文件大小限制和 CSV 内容校验，并避免将真实数据库密码提交到代码仓库。

10 部署设计

部署模式	数据库	启动方式	适用场景
开发模式	MySQL	mvn spring-boot:run	本机开发和调试。

便携本地模式	H2	start-anywhere.ps1 或 local profile	无 MySQL 环境的课堂演示。
打包运行模式	MySQL/H2	mvn package 后 java -jar	稳定启动和交付演示。
Docker 部署	MySQL 容器	docker compose up -d -- build	云服务器或多人访问环境。

11 测试设计

项目已按控制层、业务层、数据访问层和领域模型组织测试，测试目标如下：

测试类	覆盖内容	验证重点
WebControllerTest	页面路由和模板返回	主要页面可访问。
OperationsControllerTest	运行洞察页面和导入导出	CSV、冲突和扩展页面可用。
DomainModelDefaultsTest	实体默认值和生命周期回调	状态默认值、时间字段初始化。
RepositoryQueryTest	Repository 查询	筛选和排序正确。
BasicCrudServiceTest	基础 CRUD 服务	校验、保存和删除保护。
SimulationServiceTest	仿真核心流程	资源分配、延误、完成和重置。
StatisticsServiceTest	统计汇总	数量、比例和延误指标正确。

12 风险与扩展设计

风险/限制	影响	控制或扩展方案
调度算法简化	不能代表真实空管决策	明确课程边界，后续可加入优先级评分、天气权重和紧急航班规则。
实体间以逻辑关联为主	数据库层约束较弱	正式化版本补充外键、级联策略和迁移脚本。
登录功能为演	缺少真实认证授权	引入 Spring Security 和角色权限。

示级		
资源占用时长 等于时间步长	运行模型较粗粒度	后续可为不同机型/航班类型配置服务时长。
内存排序处理 待调度航班	大数据量时性能下降	使用数据库分页、索引和调度队列表。

13 结论

本概要设计采用分层架构和清晰的领域模型，将机场航班运行仿真拆分为基础数据管理、资源管理、仿真调度、日志追踪、统计展示和运行洞察等模块。核心调度逻辑集中在 `SimulationService` 中，数据结构围绕 `flight`、`runway`、`gate`、`simulation_task` 和 `simulation_record` 五张核心表展开。该设计能够支撑当前课程项目的代码实现、测试验收和演示答辩，也为后续扩展更复杂的调度算法、权限体系和运行分析能力保留了明确演进路径。